

CPS 2231 Computer Programming

Chapter 1: Basics of Java Programming (Review)

Dr. Hamza Djigal

Wenzhou-Kean University

Date: Fall Semester, 2025-09-01



温州肯恩大学
WENZHOUCHEAN UNIVERSITY

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

7 Methods in Java

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

7 Methods in Java

Objective

After this chapter, you should be able to understand:

- Java main features
- Java Components
- Java Syntax
- Differences between Instance variables and Static variables
- Methods syntax and declaration in Java

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

7 Methods in Java

1. Java features

1.1 Platform Independent

- Programs developed in one machine can be executed in another machine.
- Write code once and run it on almost any computing platform.

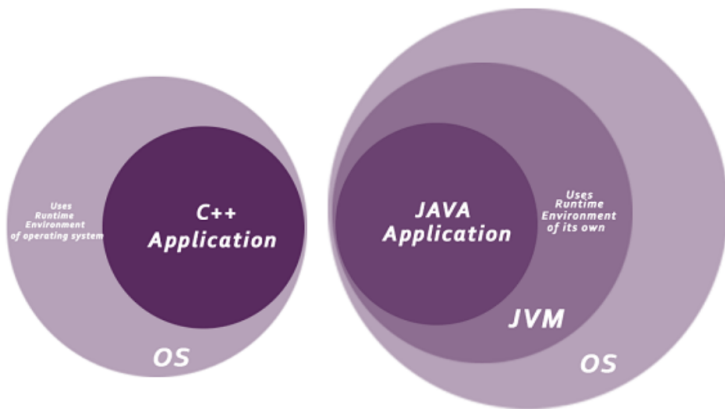
1.2. Object-Oriented Programming (OOP)

- Java is an OOP language, which simplifies software development and maintenance based on some rules.
- OOP concepts: Object, Class, Inheritance, Polymorphism, Abstraction, Interface, and Encapsulation.

1.3 Java is Secured

- No explicit pointer Java Programs run inside a **Java Virtual Machine (JVM)**. Security Manager determines the resources accessible by a class.

Java is secured



1. Java features

1.4 Java is Robust

- Java provides automatic memory management (by the help of garbage collection) which runs on the JVM to get rid of objects which are not being used by a Java application anymore.
- Java allows exception handling and the type checking mechanism.

1.5 High-performance

- Java is faster than other traditional interpreted PL because Java bytecode is "close" to native code.
- Java is a little bit slower than a compiled language (e.g., C++) because Java is an interpreted programming language.

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

7 Methods in Java

Java Platforms

There are four different types of Java programming language platforms:

- **Java Standard Edition (Java SE)** Java SE's API offers the Java programming language's core functionality. It is used for networking, security, database access, graphical user interface (GUI) development, and XML parsing.
- **Java Enterprise Edition (Java EE):** offers an API and runtime environment for developing and running highly scalable, large-scale, multi-tiered, reliable, and secure network applications.
- **Java Micro Edition (Java ME):** offers an API and a small-footprint virtual machine running Java programming language applications on small devices, like mobile phones.
- **Java FX:** JavaFX is a platform for developing rich internet applications using a lightweight user-interface API.

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

7 Methods in Java

Java Components

The following are three main Java platform components:

Java Development Kit (JDK):

- JDK is a software development environment used for making Java applications.
- Java developers can use it on Windows, macOS, Solaris, and Linux.
- It is possible to install more than one JDK version on the same computer.

Java Runtime Environment (JRE)

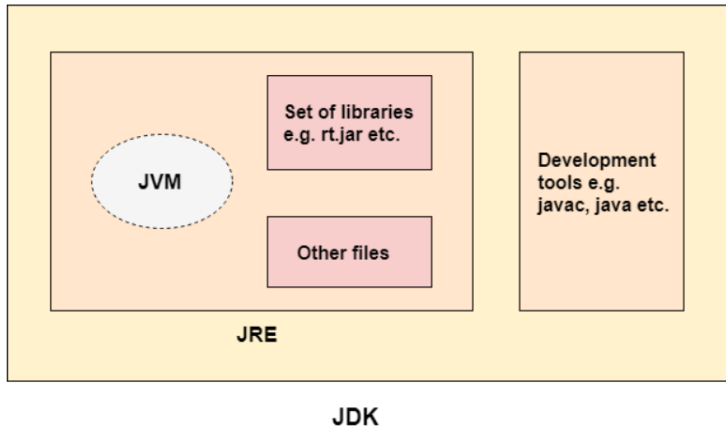
- JRE is a piece of software that is designed to run other software.
- JRE contains the class libraries, loader class, and JVM.
- If you want to run a Java program, you need JRE. You can just install JRE to run Java programs.

Java Components

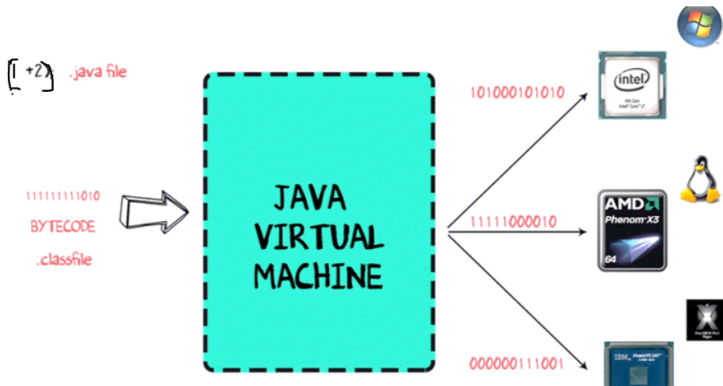
Java Virtual Machine (JVM)

- JVM is an abstract machine also called as virtual machine because it doesn't physically exist.
- JVM is platform dependent: i.e., JVMs are available for many hardware and software.
- JVM is an engine that provides a runtime environment to run the Java Code or applications.
- JVM converts Java bytecode into machine language.
- JVM comes with JIT (Just-in-Time) compiler that converts Java source code into low-level machine language. Hence, it runs faster than a regular application.
- JVM is a part of Java Runtime Environment (JRE)

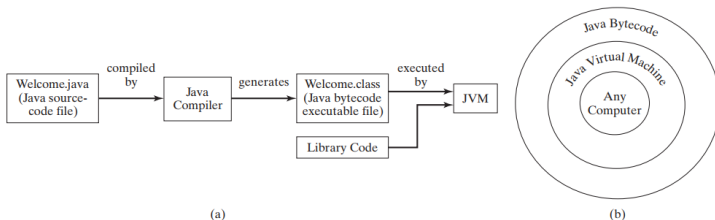
Java Components



How Java Virtual Machine works?



How Java Virtual Machine works?



Java source code is compiled into Java bytecode and Java bytecode is interpreted by the JVM.

Interpreted and Compiled Language

Compiled languages

Compiled languages are converted directly into machine code that the processor can execute. Ex. C, C++, Erlang, Haskell, Rust, and Go. More faster and more efficient to execute than interpreted languages.

Interpreted Languages

Interpreted Languages: read and execute the program at hand instruction by instruction. After being read, each instruction is translated into the machine's binary code and run.

Note

Java is both Interpreted and Compiled Language

Interpreter vs Compiler

Interpreter	Compiler
Processes programs one instruction at a time.	Processes whole program at once
Executes programs by loading and translating instructions one by one on the fly.	Translates programs to binary machine code.
Runs each time the program is executed	Need only once after the program is completed.
All errors are caught during execution	Allows for detection of some errors prior to execution.

Practice 1: Setting Java Environment

See the Attached Lab1 file in Canvas.

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

7 Methods in Java

Java Basic

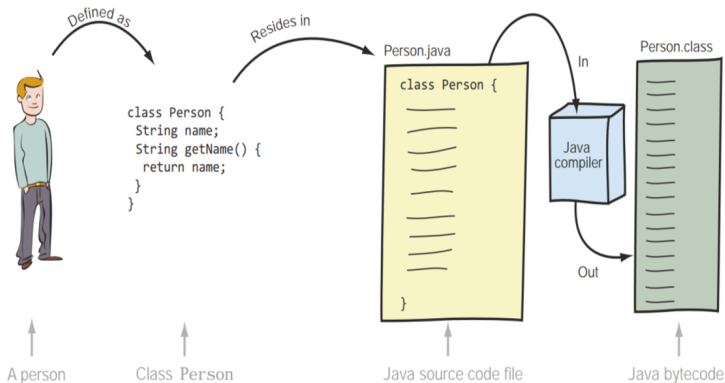


Fig. 1: Relation Between class Person, Person.Java, and Person.class

Structure of a Java class

```
Package statement
Import statements
Comments
Class declaration {
    Variables
    Comments
    Constructors
    Methods
    Nested classes
    Nested interfaces
    Enum
}
```

Fig. 2: Structure of a Java Class

Package in Java

- All Java classes are part of a package.
- Java class can be explicitly defined in a named package; otherwise, it becomes part of a default package, which doesn't have a name.
- The package statement can't appear within a class declaration or after the class declaration.
- Classes in the same package can use each other without prefixing their names with the package name.
- But to use a class from another package, you must use its fully qualified name, that is,
`packageName.anySubpackageName.ClassName`.

Package in Java

- For example, the fully qualified name of class String is java.lang.String
- You can use the import statement to use the simple name of a class in your code.



```

package university;
import certification.ExamQuestion;
class AnnualExam {
    ExamQuestion eq;
}
  
```

Define a variable
of ExamQuestion

Snipping Tool

Fig. 3: Java Package

Package in Java

- If a package statement is present in a class, the import statement must follow the package statement.
- It's important to maintain the order of the occurrence of the package and import statements.

```
import certification.ExamQuestion;  
package university;  
class AnnualExam {  
    ExamQuestion eq;  
}
```



The code won't compile because an import statement can't be placed before a package statement.

Fig. 4: Java Package

Class declaration components

- A class name starts with the keyword `class`. You can't use the word `Class` (uppercase C) to define a class.
- The state of a class is defined using attributes or instance variables.
- It isn't compulsory to define all attributes of a class before defining its methods.
- The behavior is defined using methods, which may include method parameters.
- A class definition may also include comments and constructors.

Class declaration components

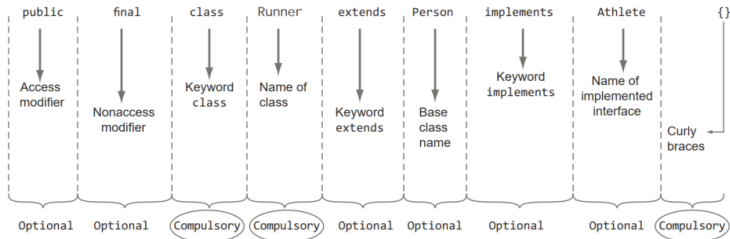
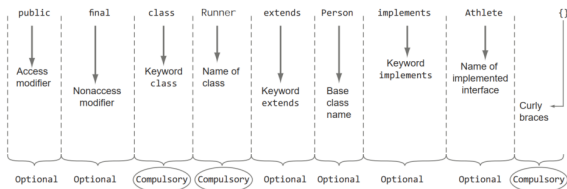


Fig. 5: Class declaration components

Java main Method

Java main method should comply with the following rules:

- A class name starts with the keyword class. You can't use the word Class (uppercase C) to define a class.
- The state of a class is defined using attributes or instance variables.
- It isn't compulsory to define all attributes of a class before defining its methods.
- The behavior is defined using methods, which may include method parameters.
- A class definition may also include comments and constructors.



Java main Method

1) It's valid to define the method parameter passed to the main method as a variable argument (varargs) of type String:

```
1 public static void main(String... args){ }  
2 // It is valid declaration  
3
```

Note

2) To define a variable argument, the ellipsis (...) must follow the type of the variable and not the variable itself (a mistake made by a lot of new programmers):

```
5 public static void main(String args...){ }  
6 // This will not compile  
7
```

Java main Method

Variable arguments or varargs: In java, a method can accept arbitrary number of values. Therefore, This argument is called varargs.

```
1 //Example without using varargs
2 class NoVararg {
3     public int sumNumber(int a, int b){
4         return a+b;
5     }
6     public int sumNumber(int a, int b, int c){
7         return a+b+c;
8     }
9     public static void main( String[] args ) {
10         NoVararg obj = new NoVararg();
11         System.out.println(obj.sumNumber(1, 2));
12         System.out.println(obj.sumNumber(1, 2, 3));
13     }
14 }
```

Java main Method

```
1 //Example with using varargs
2 class VarargExample {
3     public int sumNumber(int ... args){
4         System.out.println("argument length: " +
5             args.length);
6         int sum = 0;
7         for(int x: args){
8             sum += x;
9         }
10        return sum;
11    }
12    public static void main( String[] args ) {
13        VarargExample ex = new VarargExample();
14        int sum2 = ex.sumNumber(2, 4);
15        System.out.println("sum2 = " + sum2);
16        int sum3 = ex.sumNumber(1, 3, 5);
17        System.out.println("sum3 = " + sum3);
18    } }
```

Java main Method

3) In the following example the names of a method argument is HelloWorld, which is valid.

```
18  
19 public static void main(String[] HelloWorld){ }  
20 // It is valid declaration
```

4) The **square brackets []** can follow either the variable name or its type.

```
22  
23 public static void main(String HelloHD[]){ }  
24 // It is valid
```

5) The **placement of the keywords public and static are interchangeable**

```
25  
26 public static void main(String[] args){ }  
27 static public void main(String[] args){ }
```

Java Packages

The need for packages: use a package to group together a related set of classes and interfaces. **Rules to remember about packages:**

- Package names should all be in lowercase.
- The package and subpackage names are separated using a dot (.).
- Package names follow the rules defined for valid identifiers in Java.
- For classes and interfaces defined in a package, the package statement is the first statement in a Java source file (a .java file). The exception is that comments can appear before or after a package statement.
- There can be a maximum of one package statement per Java source code file (.java file).
- All the classes and interfaces defined in a Java source code file are defined in the same package.

My java code

```
1 public class HelloWorld
2 {
3     public static void main(String[]
         args)
4     {
5         System.out.println("Hello World"
            );
6     }
7 }
```

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

7 Methods in Java

Java Variables

Local Variables:

- A local variable is a variable defined within a block or method.
- It is mandatory to initialize a local variable before using it in the defined scope.

```
1 public static void main(String[] args){  
2     // Declare a local variable var1  
3     int var1 = 10;  
4  
5     //The variable var1 is local to the main method  
6     only  
7 }
```

Java Variables

Instances Variables:

- Instance variables are declared in a class outside of any method or block.
- Instance variables are created when an object of the class is created and destroyed when the object is destroyed.
- Initialization of an instance variable is not mandatory. Its default value is dependent on the data type of variable. For String it is null, for float it is 0.0f, for int it is 0
- **Instance variables can be accessed** only by creating objects.

Static Variables or Class Variables

- Static variables are declared similarly to instance variables, but are declared using the static keyword.
- static variables cannot be declared locally.

Differences Between the Instance Variables and the Static Variables

Instance variables	Static variables
Each object will have its own copy of an instance variable.	One copy of a static variable per class.
Can access instance variables through object references.	Can be accessed directly using the class name.
Created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.	Created when the program starts and destroyed when the program stops.

1 Learning Objectives

2 Java Features

3 Java Platforms

4 Java Components

5 Java Basic Syntax

6 Java Variables

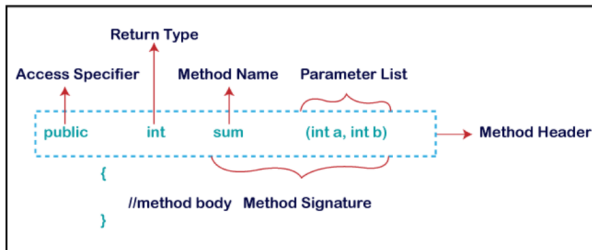
7 **Methods in Java**

Method in Java

Definition:

- **Method** in Java is a collection of instructions that performs a specific task.
- A method is a block of code or collection of statements grouped together to perform a certain task or operation.
- It provides the reusability of code. Help to modify code using methods.

Method Declaration



Exercise 1

The following java program has errors that prevent the program from compiling. Find and fix the errors.

```
public class Practice_02
{
    public static void main(String[] args)
    {
        System.out.println("Enter the length and width of rectangle");
        System.out.println("-to calculate its area");
        Scanner keyboard = new Scanner(System.in);
        int l = keyboard.nextInt();
        int w = keyboard.nextInt();
        int area = l * w;
        System.out.println("The area of the rectangle is " area);
    }
}
```